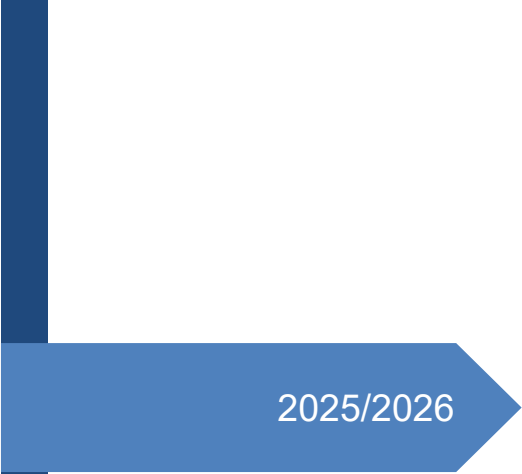
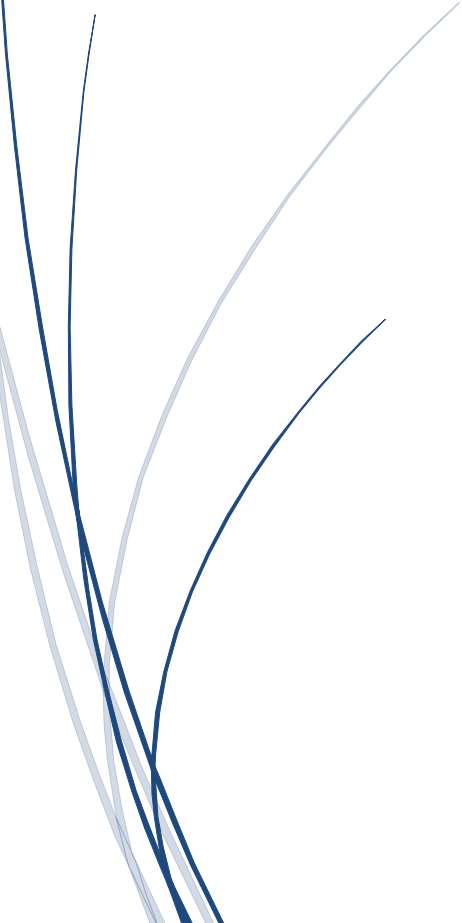




2025/2026

ISW2: Apache Syncope

Software Testing



Matteo Pinfildi
MATRICOLA: 0376255

Sommario

1. Introduzione e Setup Sperimentale	1
2. Manual Testing.....	1
2.1. Valutazione Test BB con JaCoCo	4
2.2. Valutazione Test BB con PIT	6
3. Randoop	7
4. Generazione dei Test tramite LLM.....	8
5. EvoSuite.....	10
6. Stima della Reliability.....	11
7. Analisi e valutazione del Refactoring Automatico.....	12
A. Tabelle.....	16
A.1. Tabelle Manual Testing	16
A.2. Tabelle Generazione Automatica	18
A.3. Tabelle sulle versioni Refactorizzate.....	19
B. Prompt per LLM.....	20

1. Introduzione e Setup Sperimentale

Il progetto in esame è Apache Syncope, un framework open-source sviluppato da Apache per la gestione delle identità e degli accessi digitali. Per garantire il totale isolamento delle sperimentazioni e per non intaccare il repository originale, l'ambiente di lavoro è stato configurato partendo da una procedura di fork del progetto primario tramite l'interfaccia di GitHub; successivamente, il repository remoto è stato scaricato localmente nell'ambiente di lavoro.

Al fine di automatizzare il processo di validazione del codice e certificare l'integrità della build a ogni operazione di push, è stato configurato un ambiente di Continuous Integration basato su GitHub Actions. Essendo Syncope un progetto multi-modulo di ampie dimensioni, il repository sottoposto a fork ha inizialmente ereditato l'intera suite di pipeline CI/CD native di Apache. Tuttavia, mantenere tali configurazioni avrebbe comportato l'avvio automatico di molti controlli di qualità e test globali, generando un inutile overhead computazionale e inevitabili fallimenti a cascata. Per prevenire questo scenario, si è proceduto con l'eliminazione di tutti i file YAML nativi di Apache. Successivamente, è stato aggiunto un singolo file YAML personalizzato (`maven.yml`), creando una pipeline isolata e dedicata esclusivamente agli scopi di questo esperimento.

La sperimentazione è stata condotta su due classi appartenenti ai moduli core di Apache Syncope: `ResourceLogic` e `DefaultRealmPullResultHandler`. La prima classe appartiene al package `org.apache.syncope.core.logic` e si occupa di gestire le "Risorse Esterne". Una Risorsa Esterna rappresenta un qualsiasi sistema esterno a cui Syncope deve collegarsi. Questa classe si occupa di creare, leggere, aggiornare o eliminare le configurazioni di queste connessioni, assicurandosi che i dati siano validi, prima di salvarli nel sistema. La seconda classe appartiene al package: `org.apache.syncope.core.provisioning.java.pushpull` ed è responsabile della gestione dei risultati di sincronizzazione durante le operazioni di Pull; questo flusso si verifica quando Syncope interroga una Risorsa Esterna per rilevare eventuali modifiche di stato, definite come delta, al fine di aggiornare i propri dati. La classe analizza i dati in ingresso e applica le modifiche richieste all'interno dei Realm, ovvero i contenitori logici con cui Syncope definisce la gerarchia aziendale.

Per l'implementazione pratica del progetto sono stati utilizzati tre differenti strumenti per la generazione dei test: Copilot, Randoop ed EvoSuite. Per la validazione delle suite prodotte si sono utilizzati: JaCoCo e PIT, rispettivamente per la code coverage e il mutation score. Tutto il materiale sviluppato è disponibile sul seguente repository GitHub:

<https://github.com/matteopinildi/syncope>

2. Manual Testing

L'attività di manual testing è stata condotta adottando un approccio di tipo Black-Box, ignorando dunque l'implementazione del codice sorgente e concentrandosi principalmente su interfacce pubbliche, firme dei metodi e sulla documentazione ufficiale (*Syncope Reference Guide*). Per `ResourceLogic` sono state selezionate le operazioni legate alla creazione logica della risorsa e all'interrogazione dei sistemi remoti, mentre per `DefaultRealmPullResultHandler` sono stati scelti i metodi decisionali ed esecutivi relativi alla gestione degli eventi di sincronizzazione e al provisioning dei Realm.

Per la progettazione dei test manuali si è adottata la tecnica del Category Partition. I parametri di input dei metodi presi in esame sono stati analizzati per individuare le partizioni valide e non valide, estraendo da queste i valori rappresentativi per l'esecuzione. I test case finali sono stati definiti rispettando i seguenti vincoli:

- prevedere almeno un test eseguito utilizzando esclusivamente parametri validi;
- scrivere almeno un test mirato per ogni singola partizione non valida identificata;

- inserire un solo input scorretto alla volta all'interno dei test di fallimento, in modo da isolare chiaramente l'origine dell'errore;
- assicurare che la totalità delle partizioni individuate sia coperta da almeno un caso di test.

Per ResourceLogic sono stati selezionati i metodi: create e searchConnObjects, poiché gestiscono direttamente le comunicazioni verso i sistemi esterni. Il primo metodo, create, ha il compito di validare e salvare nel database i parametri di connessione di una nuova Risorsa. Il secondo metodo, searchConnObjects, sfrutta quella connessione per interrogare il sistema remoto. In Apache Syncope, il corretto funzionamento di questa coppia di operazioni è fondamentale: create definisce come il framework si collega alle risorse esterne, mentre searchConnObjects dimostra che il sistema riesce effettivamente a leggere i dati da esse.

Metodo 1: public ResourceTO create (final ResourceTO resourceTO). Questo metodo crea una nuova risorsa esterna all'interno del database di Syncope, a partire da un DTO fornito in input. Si osservano quindi le seguenti partizioni:

- resourceTO: {oggetto non nullo}, {null};
- key: {stringa valida e non esistente nel DB}, {stringa valida ma già esistente nel DB}, {stringa vuota}, {null};
- connector: {riferimento a connettore esistente}, {riferimento a connettore inesistente}, {null}.

Al fine di evitare un'esplosione combinatoria dei casi di test, i campi opzionali del DTO sono stati mantenuti a valori di default. L'analisi si è concentrata esclusivamente su due parametri dell'oggetto in input: key e connector. La scelta è motivata dalle definizioni formali riportate nella *Syncope Reference Guide*: il connector rappresenta il prerequisito funzionale assoluto, poiché lo scopo di una External Resource è esattamente quello di contenere le configurazioni necessarie per permettere al framework di utilizzare un determinato connettore durante lo scambio dei dati. La key rappresenta la chiave primaria dell'entità nel sistema, senza la quale nessun salvataggio è possibile.

Per resourceTO, si è distinto il caso valido dal caso null, così da verificare se il metodo applichi controlli di programmazione difensiva prima di estrarre i dati dal DTO. Nel caso di key, la stringa vuota è stata distinta da null perché rappresenta un tipo differente di partizione invalida: la stringa vuota indica un valore sintatticamente corretto, ma semanticamente incorretto per il dominio, mentre null indica un'assenza di riferimento, dunque un tipo di input invalido sia sintatticamente che semanticamente. È stata inoltre isolata la partizione della stringa valida, ma già esistente, per testare il vincolo di unicità imposto dal dominio. Per connector, oltre al caso valido, sono stati distinti i casi di assenza del riferimento (null) e di un riferimento formalmente corretto, ma inesistente nel DB, al fine di valutare se il metodo garantisce l'integrità relazionale, ovvero l'obbligo di associare la risorsa a un connettore reale.

Dalle partizioni individuate sono stati scelti i seguenti input rappresentativi:

- resourceTO: {resourceTO valido}, {null};
- key: {"Resource_DB"}, {"Resource_DB_Duplicato"}, {""}, {null};
- connector: {"Conn-123"}, {"Conn-FANTASMA"}, {null}.

I casi di test progettati per create sono riportati nella Tabella 1.

Metodo 2: Pair<SearchResult, List<ConnObject>> searchConnObjects (Filter filter, Set<String> moreAttrsToGet, String key, String anyTypeKey, int size, String pagedResultsCookie, List<Sort.Order> sort). Questo metodo esegue la lettura dei dati dalla risorsa esterna, dimostrando se la connessione è realmente operativa. Consultando le specifiche fornite dalla documentazione ufficiale, si è stabilito un criterio per la scelta dei parametri da testare. L'analisi ha isolato key, anyTypeKey e size come unici input funzionalmente indispensabili: i primi due identificano la risorsa da cui leggere e l'entità da cercare, mentre size è fondamentale per limitare il volume dei dati scaricati. Al contrario, tutti gli altri parametri si comportano come

semplici modificatori opzionali: se mantenuti ai valori di default, il metodo esegue comunque la ricerca con successo, restituendo l'intero set di dati senza specifiche formattazioni. Di conseguenza, per evitare un'inutile esplosione combinatoria dei test case, che non avrebbe aggiunto valore alla verifica della robustezza, l'analisi si è concentrata esclusivamente su quegli specifici input. Si osservano, dunque, le seguenti partizioni:

- key: {stringa valida di risorsa esistente}, {stringa valida di risorsa inesistente}, {stringa vuota}, {null};
- anyTypeKey: {stringa valida e mappata sulla risorsa}, {stringa valida ma non mappata sulla risorsa}, {stringa vuota}, {null};
- size: {negativo}, {zero}, {positivo}.

Sia per key che per anyTypeKey, le partizioni coprono i casi limite di assenza del dato (null) e di input vuoto (""). Inoltre, per i casi validi, sono stati introdotti test per valutarne la validità semantica rispetto allo stato del sistema: per key si verifica il caso in cui la risorsa sia inesistente nel database, mentre per anyTypeKey si verifica il caso in cui il tipo di entità esista nel framework ma non sia mappato sulla specifica risorsa interrogata. Per size sono stati scelti un valore negativo, zero ed un valore positivo, poiché il parametro rappresenta un vincolo numerico per la dimensione della pagina.

Dalle partizioni individuate sono stati scelti i seguenti input rappresentativi:

- key: {"Resource_DB"}, {"Resource_DB_Fantasma"}, {""}, {null};
- anyTypeKey: {"USER"} (mappato), {"GROUP"} (non mappato), {""}, {null};
- size: {-1}, {0}, {10}.

Durante l'esecuzione delle test suite progettate su questi input, l'osservazione del sistema ha permesso di far emergere un comportamento non documentato. Nello specifico, fornendo un parametro di paginazione (size) negativo o pari a zero, l'oracolo iniziale prevedeva un fallimento dell'operazione. Il sistema, tuttavia, gestisce l'input anomalo senza sollevare alcuna eccezione, limitandosi a restituire una lista vuota. Tale comportamento è stato integrato negli oracoli definitivi.

I casi di test progettati per searchConnObjects sono riportati nella Tabella 2.

Per DefaultRealmPullResultHandler sono stati selezionati i metodi handle e provision. Questa scelta deriva dallo scopo principale della classe, ovvero importare i dati provenienti dai sistemi esterni per sincronizzare i Realm. Il metodo handle costituisce l'entry-point decisionale: ha il compito di analizzare l'evento in ingresso e di determinare il corretto flusso di elaborazione. Il secondo metodo, provision, ha il compito di convertire le informazioni esterne nel formato interno di Syncope per poi delegare al metodo create il salvataggio sul database.

Metodo 1: boolean handle (SyncDelta delta). Questo metodo analizza l'evento di sincronizzazione esterno per determinare se e come debba essere gestito dal sistema. Sebbene l'oggetto SyncDelta incapsuli molteplici attributi, l'analisi per la progettazione dei test si è focalizzata esclusivamente sui due parametri che determinano il comportamento atteso del metodo: il tipo di azione da compiere, ovvero deltaType, e i dati su cui compierla, ovvero ConnectorObject. Gli altri parametri sono stati esclusi dal perimetro del test in quanto ininfluenti sull'esito dell'operazione. Per evitare di moltiplicare inutilmente i casi di test, questi campi sono stati semplicemente valorizzati con dei dati fittizi validi. Si osservano quindi le seguenti partizioni:

- SyncDelta: {oggetto non nullo}, {null};
- deltaType: {CREATE}, {UPDATE}, {DELETE};
- ConnectorObject: {oggetto non nullo}, {null}.

Per delta, si è semplicemente distinto il caso in cui l'evento è assente da quello in cui è valido. Per il tipo di operazione, sono state considerate le tre azioni di base previste dal sistema: creazione, aggiornamento e cancellazione. Infine, per quanto riguarda i dati effettivi da sincronizzare, si è testato il caso in cui sono presenti e il caso in cui non sono presenti.

I casi di test progettati per handle sono riportati nella [Tabella 3](#).

Metodo 2: `protected OpEvent.Outcome provision (final SyncDelta delta, final OrgUnit orgUnit)`. Mentre handle decide l'operazione da compiere, `provision` esegue il lavoro pratico: prende i dati dal `SyncDelta`, li adatta alle regole di traduzione definite nell'`OrgUnit` e infine delega al metodo `create` il salvataggio definitivo nel database di Syncope. Si osservano le seguenti partizioni:

- `SyncDelta`: {oggetto non nullo}, {null};
- `OrgUnit`: {oggetto non nullo}, {null};
- `ConnectorObject`: {oggetto non nullo}, {null};
- Esito persistenza: {successo}, {eccezione di sistema}.

Per i parametri `SyncDelta` e `OrgUnit` è stata mantenuta la distinzione tra oggetti validi e valori nulli. Per quando riguarda i dati effettivi da sincronizzare, ovvero il `ConnectorObject`, si è distinto il caso in cui i dati siano correttamente valorizzati da quello in cui i dati risultano essere nulli. Questa scelta deriva dal fatto che un oggetto `SyncDelta` formalmente valido, ma privo dei dati al suo interno, rende impossibile l'esecuzione del provisioning. Infine, per valutare la robustezza del metodo, è stata modellata una partizione relativa all'esito della persistenza, simulando un fallimento al fine di verificare la corretta gestione delle eccezioni di sistema.

Dalle partizioni individuate sono stati scelti i seguenti input rappresentativi:

- `SyncDelta`: {delta valido}, {null};
- `OrgUnit`: {orgUnit valido}, {null};
- `ConnectorObject`: {ConnectorObject valido}, {null};
- Esito persistenza: {Salvataggio avvenuto con successo}, {RuntimeException}.

I casi di test progettati per `provision` sono riportati in [Tabella 4](#).

2.1. Valutazione Test BB con JaCoCo

Al fine di validare la qualità e l'efficacia dei test BB, sono state selezionate due metriche, calcolate tramite JaCoCo: Statement Coverage e Branch Coverage. La prima misura la percentuale di istruzioni eseguibili valutate almeno una volta durante l'esecuzione dei test, la seconda misura la percentuale dei percorsi decisionali esplorati in tutte le loro alternative.

Osservando le metriche a livello di classe, i risultati si presentano contenuti, infatti, `ResourceLogic` ha uno statement coverage del 24%, e un branch coverage del 16%, invece `DefaultRealmPullResultHandler` ha uno statement coverage del 23%, e un branch coverage del 15%. È fondamentale osservare che tali metriche, calcolate tramite JaCoCo, misurano la copertura strutturale, dunque la percentuale di istruzioni eseguite, mentre l'esperimento è stato condotto adottando un approccio di testing funzionale Black-Box.

Sui singoli metodi oggetto di studio, la coverage risulta essere nettamente più significativa. In `ResourceLogic`, il metodo `create` ha il 100% di statement e branch coverage, a dimostrazione di come le partizioni abbiano stimolato esaustivamente i controlli strutturali del metodo, al contrario `searchConnObjects` resta più basso, infatti, mostra il 60% di statement coverage e il 25% di branch coverage. In `DefaultRealmPullResultHandler`, abbiamo che: `provision` ha il 50% di statement coverage e il 40% di branch coverage, mentre `handle` mostra il 38% di statement coverage e il 33% di branch coverage. Questi risultati sono mostrati nella [Tabella 5](#).

I risultati ottenuti offrono uno spunto critico sull'allineamento tra la documentazione e l'implementazione reale all'interno di questo specifico perimetro di test. Le metriche parziali ottenute su `handle`, `searchConnObjects` e `provision` evidenziano come la tecnica delle Category Partition, dedotta in modo BB, non sia riuscita a innescare tutti i percorsi esecutivi. Questo avviene perché l'implementazione interna di questi metodi contiene rami decisionali non esplicitati nella documentazione. Al contrario, il 100% di coverage registrato sul metodo `create` dimostra una perfetta sovrapponibilità: il codice sorgente è composto da una rigorosa

sequenza di validazioni esplicite, e le partizioni dedotte in logica Black-Box si sono rivelate perfettamente adeguate a mappare e innescare ogni singolo controllo strutturale.

I report generati da JaCoCo, dunque, non indicano che i test BB sono errati o poco esaustivi, ma che sono perfettamente coerenti con il loro obiettivo metodologico: verificare rigorosamente il comportamento dedotto dalla documentazione e validare la capacità della classe di produrre i risultati attesi senza esporre errori imprevisti.

Conclusa la valutazione della baseline JaCoCo, si è proceduto alla fase di ottimizzazione manuale, dunque, si è usato un approccio White-Box. Tale attività si è concentrata sui metodi oggetto di studio: l'obiettivo era quello di esplorare i rami implementativi non emersi durante la progettazione Black-Box.

Raffinamento di ResourceLogic: per questa classe, l'intervento White-Box ha mirato a coprire le lacune strutturali del metodo searchConnObjects. L'analisi del Control-Flow Graph ha rivelato che la mancanza di copertura era dovuta prevalentemente alla diramazione condizionale associata a REALM_ANYTYPE. Nei test BB, infatti, erano state testate unicamente le entità: USER e GROUP, tralasciando l'intero blocco dedicato alla ricerca dei Realm e al relativo controllo sull'OrgUnit. L'integrazione di un test dedicato ha prodotto un miglioramento delle metriche, infatti, il metodo è passato dal 60% di statement coverage al 79% e dal 25% di branch coverage al 75%. A livello globale, la classe ha registrato un lieve incremento: dal 25% al 26% di statement coverage e dal 16% al 22% di branch coverage. Il nuovo test case è riportato nella Tabella 6.

Successivamente, al fine di validare la corretta comunicazione tra la classe presa in esame e il livello di conversione dati, si è deciso di progettare un test di integrazione. Per questo scopo è stato scelto il metodo public ResourceTO read (final String key), in quanto rappresenta il caso d'uso più lineare in cui la classe funziona come un semplice intermediario, limitandosi a passare i dati da un componente all'altro senza eseguire calcoli o logiche condizionali. Tuttavia, per poter procedere con l'integration testing, è stato necessario rispettare un'assunzione teorica fondamentale: ogni modulo o unità funzionale deve comportarsi correttamente in totale isolamento prima di poter essere integrata nel sistema. Di conseguenza, prima di implementare il test di integrazione, sono stati aggiunti due test di unità per il metodo read. Dopo aver isolato la classe, si è validato sia il percorso di successo in cui la risorsa è presente nel database, sia la diramazione logica che genera una NotFoundException in caso di risorsa inesistente. I test aggiunti sono visibili in Tabella 7.

Dopo aver verificato che il singolo metodo funziona correttamente in isolamento superando i test di unità, si è potuto procedere con l'aggiunta del test di integrazione: ResourceLogicITTest. Con questo test, che coinvolge le istanze reali di ResourceLogic e ResourceDataBinderImpl, si vuole validare che i due componenti si scambino correttamente le informazioni. Nello specifico, si verifica che la logica di business:

- interroghi l'ExternalResourceDAO per verificare l'esistenza della risorsa e recuperare l'entità dal database tramite la key fornita;
- invochi il binder reale per avviare la conversione dei dati dell'entità e restituire il DTO, ovvero ResourceTO.

Il superamento del test conferma la validità del modello di interazione tra ResourceLogic e ResourceDataBinderImpl. L'integrazione di questo IT introduce nella classe un lieve incremento della statement coverage portandola al 28%, mentre la branch coverage rimane sul 22%. Le specifiche del test di integrazione sono riportate in Tabella 8.

Raffinamento di DefaultRealmPullResultHandler: sul metodo handle l'ispezione del Control-Flow Graph ha evidenziato che la mancata copertura era dovuta a specifiche diramazioni condizionali e blocchi di gestione degli errori. Nello specifico, non risultavano testate: l'interruzione del flusso per assenza dell'OrgUnit, la gestione dei flag asincroni di interruzione, ovvero stopRequested, i rami condizionali legati alla modalità di esecuzione (INCREMENTAL) e la gestione degli eventi da scartare. L'integrazione di test dedicati ha

prodotto un miglioramento delle metriche, infatti, il metodo è passato dal 28% di statement coverage al 59% e dal 33% di branch coverage al 83%. I nuovi test case sono riportati nella [Tabella 9](#).

Per quanto riguarda il metodo provision, il codice inesplorato riguardava specifiche modalità di esecuzione, come la modalità DryRun, ovvero la simulazione dell'operazione senza salvare nulla nel database, la mancata autorizzazione per eseguire la CREATE, con isPerformCreate impostato a false, e la logica di assegnazione del nodo padre a un oggetto RealmTO sprovvisto di percorso. L'integrazione di test dedicati ha prodotto un miglioramento delle metriche, infatti, il metodo è passato dal 50% di statement coverage al 90% e dal 40% di branch coverage al 80%. I nuovi test case sono riportati nella [Tabella 10](#).

A livello globale, la classe ha registrato un lieve incremento: dal 23% al 28% di statement coverage e dal 15% al 21% di branch coverage.

In entrambi i casi analizzati, l'obiettivo della fase White-Box è stato quello di intercettare le porzioni di codice sfuggite alla progettazione Black-Box, incrementando così la copertura strutturale. Tuttavia, emerge un chiaro limite intrinseco di questo approccio: l'elevata fragilità della suite; infatti, i test White-Box risultano fortemente accoppiati all'implementazione interna del sistema, di conseguenza, una futura evoluzione o un refactoring del codice, pur mantenendo inalterato il comportamento esterno, potrebbe portare al fallimento dei test, richiedendo costi di manutenzione della suite. I risultati dell'analisi con JaCoCo sono mostrati in [Tabella 11](#).

2.2. Valutazione Test BB con PIT

Lo scopo del Mutation Testing è stimare l'adeguatezza di una test suite esistente tramite alterazioni artificialmente introdotte nel SUT. In questo modo è possibile misurarne sia la potenziale capacità nel rilevare errori, sia la robustezza rispetto alle evoluzioni del codice. Coerentemente con la metodologia adottata, l'analisi di PIT ha valutato l'intera suite prodotta nelle fasi precedenti, comprensiva sia dei test Black-Box sia di quelli White-Box (Control Flow). Dato che questa analisi è rimasta confinata ai soli metodi oggetti di studio, è fisiologico che la maggior parte dei mutanti generati a livello di intera classe risulti di tipo NO_COVERAGE. Per questo motivo, l'intervento ha puntato alla riduzione dei mutanti survived, in quanto indicano la presenza di porzioni di codice regolarmente attraversate dalla suite, ma prive di asserzioni o controlli in grado di rilevarne le anomalie.

Analisi su ResourceLogic: L'esecuzione dei test manuali su ResourceLogic ha prodotto un Mutation Score globale del 22%, con 15 mutanti uccisi su 69 generati. L'analisi del report di PIT sui metodi presi in esame ha evidenziato una debolezza negli oracoli di test sul metodo searchConnObjects. In particolare, era presente un mutante survived sulla riga 378, che sopravviveva perché nessun test si assicurava che venisse effettivamente lanciata la NotFoundException quando la risorsa richiesta non esiste nel database durante una ricerca per un oggetto di tipo REALM_ANYTYPE. Per irrobustire la suite, è stato aggiunto un test mirato che sollecita questo percorso di uscita anticipata. Questo test simula l'assenza della risorsa sul database, verificando che venga sollevata esplicitamente la corretta NotFoundException. Dopo l'aggiunta di questo test, ResourceLogic raggiunge un Mutation Score del 23%, con 16 mutanti uccisi su 69 generati. I dettagli del test sono riportati in [Tabella 12](#).

Resta comunque un mutante survived in create, questo riguarda la rimozione della chiamata a securityChecks. Poiché negli unit test BB questo metodo è stato intenzionalmente svuotato tramite override, al fine di isolare e collaudare la logica della classe senza sollevare eccezioni di autorizzazione, la sua assenza non ha alcun impatto sul comportamento del sistema. L'oracolo non ha modo di accorgersi della modifica apportata da PIT, poiché il risultato finale del test rimane esattamente lo stesso.

Analisi su DefaultRealmPullResultHandler: L'esecuzione dei test manuali su DefaultRealmPullResultHandler ha prodotto un Mutation Score globale del 13%, con 21 mutanti uccisi su 164 generati. L'analisi ha evidenziato un mutante sopravvissuto alla riga 110. La mutazione modificava l'istruzione costringendo il metodo a restituire sempre true. Questo mutante sopravviveva proprio per la mancanza di test volti a coprire le casistiche di errore. Per uccidere questo mutante è stato aggiunto un test che imposta l'esito interno di doHandle a FAILURE, verificando che il metodo intercetti il fallimento e restituisca effettivamente false.

L'analisi del report PIT sul metodo provision ha evidenziato due mutanti sopravvissuti che richiedevano test più specifici. Il primo (riga 197) sopravviveva alla cancellazione della chiamata al metodo end(). Per ucciderlo, è stato aggiunto un test mirato che usa il comando verify per accertarsi esplicitamente che end() venga chiamato. Il secondo mutante (riga 212) sopravviveva alla cancellazione dell'istruzione che costruisce il percorso completo, ovvero il FullPath, dell'oggetto RealmTO. Per ucciderlo è stato introdotto un nuovo test che inietta un oggetto reale, con nome e padre definiti, e usa una assertEquals per verificare che il metodo abbia effettivamente calcolato e concatenato in modo corretto il percorso finale. I dettagli di questi due nuovi test sono riportati in [Tabella 13](#).

L'aggiunta dei nuovi test ha generato un incremento del Mutation Score dal 13% al 15%, con i mutanti uccisi che salgono da 21 a 24

Il miglioramento percentuale globale è coerente con l'intervento mirato: il mutation score passa dal 13% al 15%, con i mutanti uccisi che salgono da 21 a 24. L'importanza di questa misurazione non risiede tanto nel lieve aumento quantitativo, bensì nel suo impatto qualitativo: i test aggiunti riescono a intercettare mutazioni che simulano potenziali errori logici di programmazione, irrobustendo la verifica del comportamento interno dei metodi oggetto di studio.

I risultati delle run di PIT sono riportati in [Tabella 14](#).

3. Randoop

Randoop è uno strumento di generazione automatica che costruisce casi di test esplorando lo spazio degli input e generando delle sequenze pseudo-casuali di invocazioni a metodi pubblici della classe target. L'output di Randoop si divide in Regression Test, creati per descrivere il comportamento stabile del SUT e per superare con successo le esecuzioni future, ed eventuali Error-revealing Test che catturano scenari in cui il sistema solleva eccezioni impreviste o raggiunge stati di errore. Questi test richiedono un'analisi manuale per determinare se documentino un difetto reale di robustezza o se si tratti di falsi positivi.

Durante le prime esecuzioni di Randoop sulle due classi target, lo strumento generava suite di test che terminavano con successo, ma che riportavano uno 0% di coverage. L'analisi ha evidenziato che il tool falliva nell'istanziare il SUT a causa della forte iniezione di dipendenze. Per risolvere questo problema, è stata adottata una metodologia di generazione guidata da due classi di ausiliarie: RandoopHelperLogic e RandoopHelperHandler. Queste classi si occupano di costruire e preparare i componenti necessari, gestendo manualmente l'iniezione delle dipendenze con Mockito e configurando direttamente lo stato interno degli oggetti. Fornendo queste classi a Randoop, il tool ha finalmente avuto a disposizione oggetti validi su cui chiamare i metodi, sbloccando l'esplorazione del codice.

Per ResourceLogic il tool ha generato 435 regression test e 0 error-revealing test. Analizzando questi test con JaCoCo si è visto che la suite generata raggiunge il 16% di statement coverage e il 2% di branch coverage globale. Il contributo sui metodi target è fortemente disomogeneo: il metodo create viene raggiunto e parzialmente coperto, ma searchConnObjects rimane allo 0% di coverage. L'analisi svolta con PIT conferma questi limiti mostrando un mutation score del 14%, uccidendo 10 mutanti su 69. Andando ad analizzare più nel dettaglio i mutanti, si è evidenziato che nel metodo create i test generati da Randoop riescono a uccidere i mutanti più semplici, ma lasciano in stato NO_COVERAGE quelli nascosti dietro a condizioni logiche più rigide. In searchConnObjects, l'assenza totale di coverage lascia inalterati tutti i mutanti inseriti.

Per DefaultRealmPullResultHandler sono stati generati 265 regression test e 9 error-revealing test. Analizzando questi test con JaCoCo si è visto che la suite generata raggiunge il 7% di statement coverage e il 4% di branch coverage globale. Analizzando più nel dettaglio i metodi target si è notato che: handle raggiunge il 56% di statement coverage e il 50% di branch, mentre provision ha una coverage dello 0%. Con PIT, la suite ottiene un mutation score del 2%, uccidendo solo 4 mutanti su 164. Nel metodo handle, i test generati riescono a eliminare i mutanti più semplici, ma lasciano sopravvivere quelli legati alle condizioni logiche più complesse.

In ResourceLogic non sono stati generati error-revealing test, mentre in DefaultRealmPullResultHandler ne sono stati generati 9. Analizzando questi 9 test si è notato che si concludono tutti con una NullPointerException all'interno del metodo handle. Dall'analisi manuale del codice generato dal tool è emerso che Randoop creava semplicemente l'oggetto tramite il costruttore vuoto e subito dopo invocava il metodo handle. Tuttavia, in uno scenario reale, l'architettura di Syncope richiede che venga prima iniettato un profilo di configurazione. Poiché Randoop salta questo passaggio di inizializzazione, quando il metodo tenta di leggere i dati del profilo, trova un valore nullo e viene lanciata l'eccezione. Di conseguenza, questi fallimenti non indicano un vero bug logico, ma sono solo il frutto di un utilizzo scorretto e della classe da parte di Randoop.

L'esperimento condotto con Randoop ha dimostrato che, sulle classi analizzate, la generazione automatica pura fallisce totalmente, come mostra lo 0% di coverage iniziale, a causa dell'impossibilità di risolvere autonomamente l'iniezione di dipendenze complesse. L'introduzione delle classi Helper si è rivelata una strategia indispensabile per sbloccare l'esplorazione del codice. Tuttavia, i risultati finali dimostrano che, anche fornendo un setup architetturale corretto, il random testing presenta forti limitazioni strutturali. In primo luogo, Randoop si è dimostrato incapace di generare gli input specifici richiesti per superare i blocchi condizionali complessi. Inoltre, a causa dell'impossibilità di soddisfare le firme complesse, la logica principale di metodi come searchConnObjects e provision non viene mai realmente esplorata, rimanendo in uno stato di totale NO_COVERAGE. In secondo luogo, emerge la debolezza degli oracoli generati. Infatti, anche guardando solo le porzioni di codice effettivamente coperte (come il 56% di coverage raggiunto sul metodo handle), i test non riescono a uccidere la maggior parte dei mutanti. Randoop costruisce questi oracoli semplicemente registrando il valore restituito dal metodo al momento dell'esecuzione, senza avere alcuna conoscenza delle regole di business. Di conseguenza, i test riescono a esplorare i percorsi base del codice, ma risultano inefficaci nel rilevare alterazioni semantiche reali.

In sintesi, l'utilizzo di Randoop su queste classi ha richiesto un forte supporto manuale in fase di configurazione e, pur avendo generato moltissimi test validi, non ha prodotto una suite sufficientemente robusta da poter sostituire gli oracoli scritti manualmente. I risultati ottenuti in questa fase sono riportati in [Tabella 15](#).

4. Generazione dei Test tramite LLM

La generazione tramite LLM è stata valutata usando Copilot con due strategie: Zero-Shot e Guided Tree prompting. Lo scopo era valutare la capacità di un LLM nel generare test suite, prendendo in esame la corretta compilazione, la coerenza rispetto al SUT, l'efficacia degli oracoli e le metriche fornite da JaCoCo e PIT.

Il prompt Zero-Shot chiede al modello di costruire una test suite fornendo unicamente il codice sorgente. Per entrambe le classi, questa strategia di interazione con LLM ha prodotto test sintatticamente coerenti con i SUT, ma non immediatamente utilizzabili; di conseguenza è stata necessaria una fase di repair per correggere gli errori evidenziati. Il prompt utilizzato è consultabile in [Appendice B, Prompt 1](#).

Per ResourceLogic si osserva che i 20 test generati coprono le operazioni CRUD, i controlli di sicurezza, la gestione dei token e la lettura degli oggetti.

Andando ad analizzare i test che generavano errori di compilazione o che fallivano si è visto che: il test testGetConnObjectKeyValue falliva a livello di compilazione per via di un errore di ambiguità nel setup. Infatti l'LLM ha generato la seguente istruzione: `doReturn(anyUtils).when(anyUtilsFactory).getInstance(any())`, ma andando ad analizzare il metodo getInstance del SUT, si è notato che accetta sia argomenti di tipo AnyTypeKind, che di tipo Any. Il repair ha imposto l'uso del matcher tipizzato, richiedendo esplicitamente un `any(AnyTypeKind.class)` per permettere la compilazione.

Un'altra criticità emersa dall'analisi della suite è l'utilizzo del costrutto `Answers.RETURNS_DEEP_STUBS`. Questa configurazione consente all'LLM di aggirare la creazione dei mock intermedi necessari per simulare

le chiamate a metodi concatenati. L'uso di tale pratica è considerato una criticità perché espone i test a fallimenti in caso di futuri refactoring della classe.

Per `DefaultRealmPullResultHandler`, i 15 test generati verificano i rami di esecuzione dei metodi `handle`, `doHandle`, `assign`, `provision`, `update`, `delete` e `ignore`

Il test `testDoHandle_CreateUnmatched_Assigns` falliva perché l'oracolo generato dall'LLM pretendeva di verificare l'interrogazione del metodo `getUnmatchingRule()`, fornendo però come input una lista vuota di `Realm`. Analizzando il SUT, si è evidenziato che l'input fornito innescava un'uscita anticipata nel flusso di controllo, impedendo di raggiungere il ramo di codice contenente la chiamata da verificare. Essendo strutturalmente incoerente con il flusso reale del codice, il test forzava l'attraversamento di un percorso irraggiungibile ed è stato rimosso dalla suite.

L'analisi sul test `testHandle_MissingOrgUnit_ThrowsJobExecutionException`, che causava un fallimento, ha evidenziato un errore nelle asserzioni in caso di assenza dell'`OrgUnit`; questo falliva perché l'LLM aveva generato il test attendendo due chiamate consecutive al metodo `getOrgUnit()`. L'analisi del SUT ha rivelato che, in questo caso, il codice reale effettua l'invocazione al metodo una sola volta, restituendo `false`. L'intervento di repair ha riallineato il test modificando il matcher di verifica da `times(2)` a `times(1)`.

Il prompt Guided Tree impone al modello di procedere in modo più strutturato. Rispetto alla sperimentazione Zero-Shot, questa strategia ha prodotto test ancora più coerenti a livello sintattico con i SUT, pur richiedendo comunque una fase di repair per renderli eseguibili. Un aspetto rilevante emerso durante questa fase riguarda la complessità delle istruzioni fornite: l'elevato livello di dettaglio del prompt ha limitato la capacità generativa dell'LLM in una singola interazione, restituendo inizialmente solo 6 test per `ResourceLogic` e 5 per `DefaultRealmPullResultHandler`. È stata pertanto necessaria una seconda interazione con l'LLM per espandere adeguatamente la copertura delle suite. Il prompt utilizzato è consultabile in [Appendice B, Prompt 2](#).

Per `ResourceLogic`, la suite comprende 14 test che vanno a coprire un sottoinsieme dei metodi della classe.

L'analisi si è concentrata sui test che fallivano o che generavano errori. Il test `testCreate_BlankKey_ThrowsException` (prodotto dopo la prima interazione) falliva per via di un errore dovuto ad un tipo di ritorno errato, infatti, l'LLM aveva istruito il mock di `SyncopClientException.getElements()` a restituire un `ArrayList`, mentre il metodo originale restituisce un `Set`. Nella fase di repair è stato corretto il setup sostituendo l'inizializzazione con un `HashSet` compatibile, permettendo al test di passare con successo.

Il test `testCheck_Success` falliva a causa di una simulazione incompleta da parte dell'LLM. Il modello aveva previsto correttamente la creazione finale del `Connector`, ma aveva ommesso di mockare un passaggio intermedio fondamentale: l'invocazione del metodo `buildConnInstanceOverride`. A causa di questa mancanza, il test riceveva un valore nullo e si interrompeva. Nella fase di repair è bastato aggiungere questa singola istruzione per ripristinare il corretto flusso logico e far passare il test.

Per `DefaultRealmPullResultHandler`, la suite generata, composta da 11 test, copre un sottoinsieme dei metodi della classe. In questo caso, solo il test `testHandle_ThrowsIgnoreProvisionException_ReturnsTrue` falliva lanciando una `NullPointerException`. L'analisi del SUT ha rivelato che la logica reale, durante l'attraversamento del ramo `catch` che gestisce l'eccezione `IgnoreProvisionException`, invoca il metodo `connectorObject.getUid()` per la costruzione del log d'errore. L'LLM non aveva considerato la configurazione di tale invocazione nel setup dei mock, comportando la restituzione di un valore `null` e la conseguente NPE. L'anomalia è stata risolta durante la fase di repair aggiungendo lo stubbing mancante e fornendo un'istanza valida di `Uid`, ripristinando così la corretta esecuzione del test.

I risultati quantitativi delle suite generate dagli LLM sono riportati in: [Tabella 16](#) e [Tabella 17](#).

Per `ResourceLogic` la suite Zero-Shot raggiunge il 71% di statement coverage, il 69% di branch coverage e un mutation coverage del 51%; al contrario, la suite generata tramite Guided Tree prompting registra valori

del 40% di statement coverage, 26% di branch coverage e 32% di mutation coverage. Analogamente, per DefaultRealmPullResultHandler, la suite Zero-Shot ottiene il 37% di statement coverage, 28% di branch coverage e il 24% di mutation coverage; al contrario, la suite Guided Tree riporta il 22% di statement coverage, 21% di branch coverage e il 13% di mutation coverage.

Sebbene i risultati quantitativi siano a favore dell'approccio Zero-Shot, l'ispezione manuale delle suite ha rivelato una forte discrepanza tra le metriche di copertura e l'effettiva qualità dei test. L'elevata copertura dello Zero-Shot è una diretta conseguenza dell'elevato volume di test generati, i quali risultano però superficiali. Per riuscire ad attraversare il maggior numero di righe di codice, il modello ha fatto ricorso a configurazioni di mocking forzate: si nota infatti un ampio uso della direttiva RETURNS_DEEP_STUBS. Inoltre, come mostrano i report, i test tendono a eludere il normale flusso logico, passando appositamente dati non validi con lo scopo di far scattare subito le uscite anticipate o i blocchi di gestione delle eccezioni. Anche se testare i casi di fallimento è una pratica corretta e importante, l'LLM usa questo approccio per massimizzare la coverage, senza dover simulare un contesto di utilizzo reale. Queste pratiche permettono ai test di non fallire e di uccidere i mutanti incrociati durante l'esecuzione, ma creano suite fragili che eludono i reali vincoli di dominio anziché validare la solidità della logica applicativa.

Al contrario, il Guided Tree ha prodotto un numero inferiore di casi di test a causa della complessità delle istruzioni del prompt, registrando inevitabilmente percentuali di copertura più basse. Tuttavia, la qualità del codice generato è superiore. I test del Guided Tree evitano scorciatoie come i deep stubs, costruendo invece gerarchie di oggetti realistiche e configurando con precisione i dati in ingresso. Questo dimostra che, sebbene limitata in termini di estensione quantitativa, la suite Guided Tree si interfaccia con il SUT in modo molto più coerente e fedele all'architettura reale.

In conclusione, l'analisi delle classi in esame dimostra che valutare il codice generato da un LLM basandosi esclusivamente sulle percentuali di JaCoCo e PIT può risultare fuorviante. Nel contesto di questo studio, le metriche elevate ottenute dallo Zero-Shot non si sono tradotte in un'effettiva qualità, ma sono risultate il prodotto di test numerosi e superficiali che eludono le reali logiche di dominio. Al contrario, il Guided Tree ha dimostrato che, sebbene la complessità delle istruzioni possa limitare il volume dei test generati, andando a penalizzare la coverage complessiva, l'uso di un prompt strutturato ha permesso di ottenere test molto più aderenti al reale funzionamento del sistema. Ciò evidenzia come i soli dati quantitativi non siano sufficienti per attestare la reale validità e robustezza di una test suite generata automaticamente.

5. EvoSuite

EvoSuite è stato utilizzato come strumento di generazione automatica search-based. Questo tool sfrutta un algoritmo genetico per far evolvere iterativamente una popolazione di casi di test, guidando la generazione verso la massimizzazione della coverage strutturale.

Un problema riscontrato è che la versione presa in esame di Syncope richiede Java 25, a differenza di EvoSuite che risulta compatibile al massimo con Java 9. Tentare di eseguire EvoSuite su classi compilate con versioni più recenti causa inevitabili fallimenti a livello di JVM. Per risolvere il problema e poter valutare comunque l'efficacia del tool sulle classi in esame, le sperimentazioni sono state eseguite scaricando e configurando la versione 2.1 di Syncope, che risulta essere compatibile con Java 9. Successivamente, le suite prodotte dal tool sono state importate nel progetto principale e rieseguite sulla versione target con Java 25.

Per ResourceLogic, ES ha generato una suite di 21 test eseguibili. Effettuando un'analisi con JaCoCo, la coverage strutturale globale risulta essere fortemente limitata, raggiungendo il 12% di statement coverage ed il 41% di branch coverage. A livello dei metodi oggetti di studio, i limiti dello strumento sono ancora più evidenti. Il metodo create raggiunge il 22% di statement coverage e il 25% di branch coverage, mentre searchConnObjects si ferma al 4% di statement coverage e al 25% di branch coverage. L'analisi del codice generato rivela la causa di questa bassa copertura: ES non riesce ad istanziare le complesse dipendenze del SUT, finendo per invocare il costruttore di ResourceLogic passando esclusivamente parametri null.

Con PIT, la suite di test ottiene solo il 3% di mutation score, uccidendo 2 mutanti su 69. L'elevata presenza di mutanti in stato NO_COVERAGE certifica il limite strutturale emerso: i test generati si limitano a testare la robustezza del sistema rispetto agli input non validi. La logica di dominio profonda non viene mai fisicamente attraversata dai flussi di esecuzione.

Un caso rilevante emerso dalla suite è il test16. Il test istanzia un DTO vuoto e lo fornisce in input al metodo create, configurando l'oracolo per decretare il successo del test in caso di cattura di una RuntimeException. Non essendo in grado di costruire un DTO formalmente valido e avendo iniettato parametri nulli nel costruttore del SUT, il tool si dimostra strutturalmente incapace di oltrepassare il vincolo di validazione iniziale, precludendosi l'esplorazione della reale logica di business.

Per DefaultRealmPullResultHandler, EvoSuite ha generato un solo test. L'esecuzione della suite resituisce un fallimento totale, registrando lo 0% in: statement coverage, branch coverage e mutation score.

Analizzando l'unico test prodotto, si nota che EvoSuite istanzia la classe target tramite costruttore vuoto e si limita ad invocare il metodo handle passandogli null come parametro. Questo causa una NullPointerException alla prima riga di codice, impedendo al test di esplorare le diramazioni interne.

Il fallimento su questa classe evidenzia un limite architetturale di EvoSuite, infatti, il metodo handle non elabora tipi di dato semplici, ma richiede in input un oggetto SyncDelta caratterizzato da una struttura gerarchica complessa, dove un oggetto ne contiene altri a più livelli. In questo caso, il tool non è riuscito a comporre in modo autonomo la complessa catena di oggetti necessaria, ripiegando su input nulli e rendendo la suite generata inutilizzabile. In conclusione, EvoSuite non si è dimostrato in grado di produrre test suite capaci di sostituire integralmente l'attività di manual testing.

I risultati complessivi di queste sperimentazioni sono visibili in [Tabella 18](#).

Inizialmente si è ipotizzato di introdurre delle classi Helper anche per EvoSuite, replicando la strategia adottata con successo per Randoop. Tuttavia, un'analisi architetturale dei due tool ha evidenziato l'inapplicabilità di questo approccio: mentre Randoop accetta in input una lista di classi e genera sequenze combinandole, permettendo l'integrazione di classi di supporto, EvoSuite opera tramite una strumentazione del bytecode orientata alla singola classe target. Questo approccio, che isola la classe durante la generazione, rende di fatto inefficace l'iniezione di classi di supporto esterne.

6. Stima della Reliability

A conclusione dell'analisi, si è proceduto con la stima della Reliability delle due classi analizzate. È importante notare che la Reliability non rappresenta una prova formale di assoluta correttezza, ma solo un attributo statistico che stima la probabilità che un'interazione col sistema termini con successo.

L'insieme dei test aggregati per ResourceLogic è composto da 511 casi di test, i quali non riportano alcun fallimento. Dunque, definendo i casi di test totali come $N = 511$ ed i test falliti come $F = 0$, è stata calcolata la Probability of Failure on Demand (PFD):

$$PFD = \frac{F}{N} = \frac{0}{511} = 0$$

Dunque, la Reliability è:

$$Reliability = 1 - PFD = 1 - 0 = 1 (= 100\%)$$

Analogamente, l'insieme finale dei test per la classe DefaultRealmPullResultHandler ammonta a 310 casi di test, i quali non riportano alcun fallimento. Dunque, definendo i casi di test totali come $N = 310$ ed i test falliti come $F = 0$, è stata calcolata la Probability of Failure on Demand (PFD):

$$PFD = \frac{F}{N} = \frac{0}{310} = 0$$

Dunque, la Reliability è:

$$Reliability = 1 - PFD = 1 - 0 = 1 (= 100\%)$$

La stima della Reliability per entrambi i componenti si attesta al valore massimo. Poiché non si sono registrati fallimenti, non si è resa necessaria la disabilitazione di alcun caso di test prima della build finale. È fondamentale evidenziare, dal punto di vista dell'Ingegneria del Software, i limiti intrinseci di questa valutazione. Il raggiungimento di una Reliability pari a 1 non certifica la correttezza assoluta dei componenti: tale valore è, di fatto, un risultato statistico strettamente dipendente dall'insieme dei test assunti come campione.

7. Analisi e valutazione del Refactoring Automatico

Nell'ultima fase del progetto, è stato condotto un esperimento di refactoring automatizzato tramite LLM, adottando un approccio di prompting incrementale. Per ogni versione, sono state eseguite sessioni di generazione indipendenti, introducendo progressivamente un nuovo vincolo rispetto alla classe originale, definita come C0. Le varianti così ottenute sono state denominate C1, C2, C3 e C4. Per garantire l'isolamento dell'esperimento, ogni variante generata è stata copiata temporaneamente nel path originale del progetto, sostituendo la classe C0. Il prompt utilizzato per la generazione delle classi refactorizzate è riportato in [Appendice B](#), [Prompt 3](#).

Dopo aver generato le nuove versioni, è stata condotta un'analisi qualitativa delle differenze architetturali e sintattiche tra la versione originale C0 e le varianti sottoposte a refactoring, ovvero C1, C2, C3 e C4.

Per ResourceLogic emerge che le varianti C1 e C2 adottano un approccio estremamente conservativo. Gli interventi di refactoring si concentrano principalmente su due aspetti. In primo luogo, la classe originale presenta un costruttore a 9 parametri, violando le comuni metriche di qualità statiche che suggeriscono un limite massimo di 7. Tuttavia, modificarne la firma avrebbe compromesso la compatibilità con il resto del sistema e, a partire dalla variante C2, infranto i vincoli imposti dalle suite di test. Di conseguenza, per non alterare il comportamento del componente, l'LLM ha preferito aggirare la segnalazione applicando l'annotazione `@SuppressWarnings("java:S107")`. In secondo luogo, il code smell relativo alle stringhe letterali duplicate all'interno della classe è stato risolto estraendole in due costanti private statiche, migliorando la pulizia e la manutenibilità del codice. Inoltre, nelle varianti C1, C2 e C3 l'LLM ha risolto un code smell presente nel metodo `resolveReference`: la cattura dell'eccezione generica tramite il blocco `catch (Throwable)` è stata sostituita con l'intercettazione più specifica `catch (Exception)`.

Sulla variante C4, l'LLM applica una scomposizione strutturale decisamente più profonda. L'LLM ha identificato che i metodi `setLatestSyncToken` e `removeSyncToken` condividevano gran parte del codice; li ha quindi fusi in un nuovo metodo privato: `updateSyncToken`, che utilizza un flag booleano per decidere se aggiornare il token o impostarlo a null. Stesso discorso per il blocco di verifica di autorizzazione presente in `create`, `update`, `delete`, `setLatestSyncToken` e `removeSyncToken`, che è stato inserito in un altro metodo privato: `checkAuth`. Si nota dunque che C4 ha subito una scomposizione a livello strutturale che, pur aumentando l'invasività del refactoring, ottimizza la modularità della classe eliminandone la ridondanza.

Per quanto riguarda `DefaultRealmPullResultHandler` le varianti C1, C2 e C3 si concentrano su un problema strutturale: infatti, C0 usa l'annotazione `@Autowired` per caricare le dipendenze. Questa pratica è considerata un code smell. Per risolvere tale problema, le varianti C1 e C2 hanno rimosso queste annotazioni spostando l'inizializzazione delle dipendenze all'interno di un unico costruttore. Tuttavia, è emersa una differenza: C1 include nel costruttore solo le 7 dipendenze dirette della classe, mentre C2 estende la firma del costruttore a 15 parametri, includendo forzatamente anche le dipendenze ereditate dalla superclasse `AbstractRealmResultHandler`. Questa scelta, seppur invasiva, si è resa necessaria per permettere a tutti i test Black-Box di passare con successo. Nella variante C3, invece, l'LLM ha adottato un

approccio alternativo: ha rimosso le annotazioni `@Autowired` dai campi e ha creato dei metodi setter dedicati per caricare ciascuna dipendenza.

Oltre a questo aspetto, C1, C2 e C3 presentano altri interventi mirati alla pulizia del codice: estraggono le stringhe di testo ripetute in costanti statiche e, per abbassare la complessità logica, isolano i pesanti blocchi try-catch all'interno di nuovi metodi privati di supporto.

A differenza delle varianti precedenti, nella variante C4 l'LLM ha mantenuto inalterate le annotazioni `@Autowired`. Il modello ha preferito applicare una riorganizzazione strutturale per eliminare le ridondanze logiche e ottimizzare il codice. Avendo identificato che i metodi `assign` e `provision` condividevano gran parte delle istruzioni, per risolvere questa ridondanza, l'LLM ha fuso i due metodi all'interno del nuovo metodo privato `createRealm`. Un ulteriore intervento strutturale svolto in C4 riguarda il metodo centrale `doHandle`. Nella versione originale, quest'ultimo presentava un'eccessiva complessità dovuta a una serie di condizioni logiche annidate necessarie a smistare le operazioni. Per mitigare il problema e abbassarne la complessità ciclomatica, l'LLM ha scomposto il flusso generando due nuovi metodi ausiliari, più compatti: `processDelta` e `handleCreateOrUpdate`.

Per valutare l'impatto delle varianti refactorizzate, sono state eseguite in isolamento su ciascuna versione di entrambe le classi in esame le seguenti suite di test: Black-Box, Control Flow, Mutation-driven, LLM, Randoop, EvoSuite. Una suite è classificata come Pass esclusivamente se il 100% dei test che la compongono termina con successo e senza sollevare eccezioni impreviste; la presenza di un singolo fallimento o errore a runtime classifica l'intera suite allo stato Fail.

Come si può vedere dalla [Tabella 19](#), i risultati ottenuti sulle varianti refactorizzate della classe `ResourceLogic` mostrano che tutte le suite di test passano correttamente al 100% su tutte le varianti. Per le varianti C1, C2 e C3, il superamento totale delle suite è coerente con la natura delle modifiche: l'LLM ha infatti applicato un refactoring conservativo, senza alterare le firme dei metodi o la gestione dello stato interno.

Per quanto riguarda C4, nonostante l'LLM abbia applicato una scomposizione strutturale più profonda, introducendo i due nuovi metodi `updateSyncToken` e `checkAuth`, l'equivalenza funzionale è rimasta intatta. Tale esito è dovuto a due fattori: in primo luogo, i nuovi metodi sono stati isolati mediante il modificatore di visibilità `private`, lasciando immutato il contratto pubblico della classe; in secondo luogo, la semantica dei flussi interni e la propagazione delle eccezioni sono state fedelmente replicate.

Per quanto riguarda `DefaultRealmPullResultHandler`, i risultati sono visibili nella [Tabella 20](#).

La variante C1 risulta essere la più problematica; infatti, falliscono due test BB: `testProvision_TC01`, `testHandle_TC022`, tre test CF: `testProvision_TC05`, `testProvision_TC06`, `testProvision_TC07`, e i tre test MT: `testProvision_TC08`, `testProvision_TC09`, `testHandle_TC09`. Dall'analisi dei log di errore, emerge che tutti i fallimenti generano una `NullPointerException`. Questa anomalia è riconducibile al nuovo costruttore generato dall'LLM in C1: rimuovendo le annotazioni `@Autowired` a favore di un costruttore a soli 7 parametri, il modello ha omesso di inizializzare le dipendenze ereditate dalla superclasse. Tale limitazione strutturale causa anche il fallimento in fase di compilazione delle suite Randoop ed EvoSuite, le quali restituiscono l'errore bloccante: `Expected 7 arguments but found 0`.

Nella variante C2, l'LLM ha generato un costruttore a 15 parametri per inglobare le dipendenze ereditate. Questo approccio ha permesso il superamento delle suite manuali Black-Box e Control-Flow. Tuttavia, la variante fallisce i tre test Mutation-Driven: `testProvision_TC08`, `testProvision_TC09`, `testHandle_TC09`, scatenando la stessa eccezione osservata in C1: `NullPointerException`. Inoltre, in modo analogo a C1, la presenza del nuovo costruttore a 15 parametri causa il totale fallimento in fase di compilazione delle suite automatiche Randoop ed EvoSuite, che mostrano il seguente errore di compilazione: `Expected 15 arguments but found 0`.

Per le varianti C3 e C4, tutte le suite di test terminano con successo. Nella variante C3 l'LLM ha evitato la creazione di un costruttore esteso, introducendo dei metodi setter dedicati per il caricamento delle dipendenze. L'assenza di un costruttore esplicito a più parametri preserva l'esistenza del costruttore di default vuoto, permettendo la corretta compilazione e l'esecuzione senza fallimenti delle suite automatiche Randoop ed EvoSuite. Gli stessi risultati si registrano sulla variante C4, la quale mantiene l'annotazione `@Autowired` originaria della classe C0. Pur avendo applicato una profonda scomposizione strutturale ai metodi `doHandle`, `assign` e `provision`, l'LLM è riuscito a preservare intatto il contratto pubblico e la logica originaria della classe, garantendo il superamento di tutti i test.

La fase successiva dell'analisi mira a verificare se il refactoring abbia migliorato la testabilità del codice. Per valutare questo aspetto, il processo di generazione dei test è stato ripetuto nuovamente su ciascuna delle varianti, utilizzando Randoop e l'approccio Guided Tree basato su LLM ([Prompt 2](#)). L'efficacia di queste nuove suite è stata poi confrontata con quella delle suite originariamente prodotte per la versione C0. Per ogni confronto è stato calcolato lo scostamento rispetto all'originale, prendendo in esame cinque metriche chiave: statement coverage e branch coverage, calcolate tramite JaCoCo, mutation score e numero totale di mutanti uccisi, calcolati tramite PIT, e infine la variazione del numero di code smell. I valori completi relativi alle classi in esame sono riportati in [Tabella 21](#) e [Tabella 22](#).

Per ResourceLogic, tutte le varianti refactorizzate riducono i code smell rispetto a C0: C1 e C2 li azzerano completamente, mentre C3 e C4 li dimezzano. Tuttavia, analizzando i dati ottenuti sulle suite LLM, emerge che questo miglioramento strutturale ha portato ad un peggioramento di tutte le metriche di coverage. La statement coverage cala in tutte le versioni, toccando il picco negativo in C3, così come la branch coverage, che cala specialmente in C4. Allo stesso modo, il mutation score diminuisce per tutte le varianti, evidenziando una minore capacità della suite generata di uccidere i mutanti.

Per quanto riguarda Randoop, la branch coverage rimane al 2% in tutte le versioni; anche la statement coverage resta invariata, registrando solo un lievissimo incremento in C4. Per quanto riguarda il mutation score, le varianti subiscono un lieve peggioramento rispetto a C0.

Per DefaultRealmPullResultHandler, i delta evidenziano dati non lineari a seconda della variante utilizzata. Tutte le versioni refactorizzate riducono i code smell rispetto ai 18 smell della versione C0: C2 risulta la variante qualitativamente più pulita, con 1 solo smell rimasto, mentre C4 è quella che li riduce in misura minore, con 12 smell rimasti.

Per quanto riguarda la suite generata dagli LLM sulle classi refactorizzate, si è notato che ci sono dei comportamenti contrastanti. Le varianti C1 e C3 registrano un peggioramento in tutte le metriche. La variante C2 registra un incremento della statement coverage del 19% e un lieve aumento della branch coverage, ma non migliora il mutation score, che cala dell'1%. Questo indica che, pur attraversando più linee di codice, l'LLM non è riuscito a generare asserzioni abbastanza forti da uccidere nuovi mutanti. Al contrario, la variante C4 presenta il profilo più efficace per l'LLM: pur registrando un lieve calo nella branch coverage, migliora la statement coverage e incrementa il mutation score del 5%, riuscendo a uccidere 8 mutanti in più rispetto a C0.

Analizzando invece le suite generate con Randoop, le variazioni di coverage risultano pressoché nulle in tutte le varianti, ma si registra un peggioramento costante nel mutation score e nel numero di mutanti uccisi.

Per ResourceLogic, i risultati evidenziano un'ottima conservazione dell'equivalenza funzionale ma una penalizzazione sul fronte della testabilità. Come emerso dai test, tutte le suite passano al 100% su tutte le varianti, confermando che l'LLM è riuscito a preservare intatto il comportamento originale della classe. Tuttavia, l'analisi dei delta dimostra chiaramente che la risoluzione dei code smell, in questo contesto specifico, non ha facilitato il compito di generazione dei test. In particolare, C1 e C2 rappresentano le varianti migliori sotto il profilo dell'analisi statica, dato che sono stati azzerati gli smell, ma peggiorano la capacità dell'LLM di generare suite di test efficaci, portando a cali evidenti sia nelle metriche di coverage considerate che nel mutation score. Per quanto riguarda Randoop le metriche sono rimaste praticamente inalterate. Di conseguenza, nessuna variante risulta superiore a C0.

Per `DefaultRealmPullResultHandler` i risultati evidenziano forti compromessi tra testabilità e il mantenimento del contratto pubblico della classe. A differenza di `ResourceLogic`, in questo caso le varianti C1 e C2 hanno alterato i costruttori della classe; questa modifica causa fallimenti nelle suite manuali ed errori di compilazione nei test generati da Randoop ed EvoSuite. Al contrario, C3 e C4 superano tutte le suite. Analizzando i delta generati dall'LLM, emerge chiaramente l'efficacia della variante C4: pur presentando un numero maggiore di code smell rispetto a C1, C2 e C3, risulta l'unica capace di guidare il modello verso la generazione di test realmente più robusti. Questa variante ha infatti ottenuto il miglior incremento nel mutation score, riuscendo ad uccidere 8 mutanti in più rispetto alla versione originale C0.

Dall'analisi complessiva emerge che il refactoring automatico deve essere valutato considerando molteplici aspetti, poiché l'ottimizzazione delle metriche di analisi statica non si traduce necessariamente in una migliore qualità del sistema. Ad esempio, alcune versioni hanno migliorato la pulizia del codice pur diventando incompatibili con le suite di test esistenti, come i casi di C1 e C2 in `DefaultRealmPullResultHandler`; in altri casi, hanno ridotto i code smell a discapito della capacità di rilevare mutanti. Al contrario, una variante meno efficace nel ridurre gli smell si è rivelata l'unica in grado di migliorare il mutation score, come nel caso di C4 in `DefaultRealmPullResultHandler`.

A. Tabelle

A.1. Tabelle Manual Testing

Tabella 1: test BB per create di ResourceLogic

Id Test	resourceTO	key	connector	Risultato atteso
TC_01	valido	"Resource_DB"	"Conn-123"	key=="Resource_DB";
TC_02	null	-	-	NullPointerException
TC_03	valido	""	"Conn-123"	SyncopeClientException
TC_04	valido	null	"Conn-123"	SyncopeClientException
TC_05	valido	"Resource_DB"	"Conn-FANTASMA"	SyncopeClientException
TC_06	valido	"Resource_DB"	null	SyncopeClientException
TC_07	valido	"Resource_DB_Duplicato"	"Conn-123"	DuplicateException

Tabella 2: test BB per searchConnObjects di ResourceLogic

Id Test	key	anyTypeKey	size	Risultato atteso
TC_01	"Resource_DB"	"USER"	10	Pair != null;
TC_02	"Resource_DB_Fantasma"	"USER"	10	NotFoundException
TC_03	"Resource_DB"	"GROUP"	10	NotFoundException
TC_04	"Resource_DB"	"USER"	-1	Pair != null; List.isEmpty() == true;
TC_05	""	"USER"	10	NotFoundException
TC_06	null	"USER"	10	NotFoundException
TC_07	"Resource_DB"	"USER"	0	Pair != null; List.isEmpty() == true;
TC_08	"Resource_DB"	""	10	NotFoundException
TC_09	"Resource_DB"	null	10	NotFoundException

Tabella 3: test BB per handle di DefaultRealmPullResultHandler

Id Test	delta	deltaType	ConnectorObject	Risultato atteso
TC_01	valido	CREATE	valido	true
TC_02	valido	DELETE	valido	true
TC_03	valido	UPDATE	null	NullPointerException
TC_04	null	-	-	NullPointerException

Tabella 4: test BB per provision di DefaultRealmPullResultHandler

Id Test	delta	orgUnit	ConnectorObject	Esito persistenza	Risultato atteso
TC_01	valido	valido	valido	successo	SUCCESS
TC_02	null	valido	-	successo	NullPointerException
TC_03	valido	null	valido	successo	NullPointerException
TC_04	valido	valido	null	successo	NullPointerException
TC_05	valido	valido	valido	eccezione	RuntimeException

Tabella 5: Risultati della suite BB manuale

Classe	Test introdotti	Statement Coverage	Branch Coverage
ResourceLogic	16	24%	16%
DefaultRealmPullResultHandler	9	23%	15%

Tabella 6: Test White-Box per searchConnObjects di ResourceLogic

Id Test	Condizione	Risultato atteso
TC_10	anyTypeKey == REALM_ANYTYPE OrgUnit == null	NotFoundException

Tabella 7: Test White-Box per read di ResourceLogic

Id Test	Condizione	Risultato atteso
TC_01	authFind(key) != null	ResourceTO != null key == "Resource_DB"
TC_02	authFind(key) == null	NotFoundException

Tabella 8: test di integrazione per ResourceLogic

Test	Componente reale	Dipendenze mockate	Risultato atteso
read_IT	ResourceLogic, ResourceDataBinderImpl	ExternalResourceDAO, Dipendenze interne al binder	ResourceTO != null; key == "Resource_DB"; connector == "Conn-123";

Tabella 9: test White-Box per handle di DefaultRealmPullResultHandler

Id Test	Condizione	Risultato atteso
TC_05	OrgUnit == null	false
TC_06	stopRequested == true	false
TC_07	PullMode.INCREMENTAL	true
TC_08	UnmatchingRule.IGNORE	true

Tabella 10: test White-Box per provision di DefaultRealmPullResultHandler

Id Test	Condizione	Risultato atteso
TC_05	isPerformCreate = false	SUCCESS
TC_06	FullPath == null; Parent == null;	SUCCESS
TC_07	isDryRun = true	SUCCESS

Tabella 11: Risultati della suite White-Box manuale

Classe	Test introdotti	Statement Coverage	Branch Coverage
ResourceLogic	3+1(IT)	28%	22%
DefaultRealmPullResultHandler	7	28%	21%

Tabella 12: test mutation-guided per ResourceLogic

Metodo	Id Test	Condizione	Risultato atteso
searchConnObjects	TC_11	key = "Resource_DB_Fantasma"; anyTypeKey = REALM_ANYTYPE;	NotFoundException

Tabella 13: test mutation-guided per DefaultRealmPullResultHandler

Metodo	Id Test	Condizione	Risultato atteso
handle	TC_09	doHandle ritorna FAILURE	false
provision	TC_08	isPerformCreate == false	SUCCESS; verifica invocazione di end();
provision	TC_09	RealmTO con FullPath == null	FullPath == "Padre/Figlio"

Tabella 14: Risultati delle run di PIT

Classe	Test MT introdotti	Mutation score iniziale	Killed iniziali	Mutation score finale	Killed finali
ResourceLogic	1	22%	15/69	23%	16/69
DefaultRealmPullResultHandler	3	13%	21/164	15%	24/164

A.2. Tabelle Generazione Automatica

Tabella 15: Risultati delle suite Randoop

Classe	Regression Test	Error-Revealing Test	Statement Coverage (JaCoCo)	Branch Coverage (JaCoCo)	Mutation score (PIT)	Mutanti Killed
ResourceLogic	435	0	16%	2%	14%	10/69
DefaultRealmPullResultHandler	265	9	7%	4%	2%	4/164

Tabella 16: Risultati delle suite LLM Zero-Shot

Classe	Test	Statement Coverage (JaCoCo)	Branch Coverage (JaCoCo)	Mutation score (PIT)	Mutanti Killed
ResourceLogic	20	71%	69%	51%	35/69
DefaultRealmPullResultHandler	14	37%	28%	24%	40/164

Tabella 17: Risultati delle suite LLM Guided Tree

Classe	Test	Statement Coverage (JaCoCo)	Branch Coverage (JaCoCo)	Mutation score (PIT)	Mutanti Killed
ResourceLogic	14	40%	26%	32%	22/69
DefaultRealmPullResultHandler	11	22%	21%	13%	22/164

Tabella 18: Risultati delle suite EvoSuite

Classe	Test	Statement Coverage (JaCoCo)	Branch Coverage (JaCoCo)	Mutation score (PIT)	Mutanti Killed
ResourceLogic	21	12%	41%	3%	2/69
DefaultRealmPullResultHandler	1	0%	0%	0%	0/164

A.3. Tabelle sulle versioni Refactorizzate

Tabella 19: Esito suite C0 su ResourceLogic refactored CX

Suite	C0	C1	C2	C3	C4
Test BB	P(16/16)	P(16/16)	P(16/16)	P(16/16)	P(16/16)
Test CF+IT	P(4/4)	P(4/4)	P(4/4)	P(4/4)	P(4/4)
Test MT	P(1/1)	P(1/1)	P(1/1)	P(1/1)	P(1/1)
Test LLM (Guided Tree)	P(14/14)	P(14/14)	P(14/14)	P(14/14)	P(14/14)
Test RND	P(435/435)	P(435/435)	P(435/435)	P(435/435)	P(435/435)
Test ES	P(21/21)	P(21/21)	P(21/21)	P(21/21)	P(21/21)

P = pass; F = fail; CE = compilation error;

Tabella 20: Esito suite C0 su DefaultRealmPullResultHandler refactored CX

Suite	C0	C1	C2	C3	C4
Test BB	P(9/9)	F(7/9)	P(9/9)	P(9/9)	P(9/9)
Test CF	P(7/7)	F(4/7)	P(7/7)	P(7/7)	P(7/7)
Test MT	P(3/3)	F(0/3)	F(0/3)	P(3/3)	P(3/3)
Test LLM (Guided Tree)	P(11/11)	P(11/11)	P(11/11)	P(11/11)	P(11/11)
Test RND	P(265/265)	CE	CE	P(265/265)	P(265/265)
Test ES	P(1/1)	CE	CE	P(1/1)	P(1/1)

P = pass; F = fail; CE = compilation error;

Tabella 21: Delta di coverage, mutation score e smell per ResourceLogic

Versione	Suite	Δ Smell	Δ Statement Coverage (%)	Δ Branch Coverage(%)	Δ Mutation score(%)	Δ Mutanti killed
C1-C0	Randoop	-4	0	0	-2	-2
C1-C0	LLM	-4	-5	-11	-6	-4
C2-C0	Randoop	-4	0	0	-2	-2
C2-C0	LLM	-4	-7	-5	-4	-3
C3-C0	Randoop	-2	0	0	-2	-2
C3-C0	LLM	-2	-13	-10	-6	-5
C4-C0	Randoop	-2	+2	0	-1	-1
C4-C0	LLM	-2	-11	-16	-3	-4

Tabella 22: Delta di coverage, mutation score e smell per DefaultRealmPullResultHandler

Versione	Suite	Δ Smell	Δ Statement Coverage (%)	Δ Branch Coverage(%)	Δ Mutation score(%)	Δ Mutanti killed
C1-C0	Randoop	-16	0	0	-2	-2
C1-C0	LLM	-16	-1	-7	-3	-4
C2-C0	Randoop	-17	+2	0	-2	-2
C2-C0	LLM	-17	+19	+2	-1	-1
C3-C0	Randoop	-15	-1	0	-2	-2
C3-C0	LLM	-15	-3	-4	-1	-2
C4-C0	Randoop	-6	0	0	-2	-2
C4-C0	LLM	-6	+5	-3	+5	+8

B. Prompt per LLM

Prompt 1: Zero-Shot Prompting.

"You are an AI assistant specialized in Software Engineering and testing within a Java Enterprise environment. Your task is to generate a comprehensive JUnit 4 unit test suite for the provided class. Use the Mockito framework to isolate dependencies. Cover all public methods, aiming for high branch coverage by deducing the business rules directly from the source code. Return exclusively the Java code of the test class, enclosed between the [TEST_START] and [TEST_END] tags. SUT source code: <source_code>."

Prompt 2: Guided Tree Prompting.

"Execute a guided unit test generation process by proceeding through the following sequential steps. Do not write the final code before completing and explicitly showing your reasoning for the first 4 steps:

Step 1 - Dependency Analysis: Identify the external dependencies of the class and define exactly how they should be mocked.

Step 2 - Category Partitioning: For each public method, define the equivalence classes for the inputs and identify the boundary values.

Step 3 - Edge Cases: Identify potential failure paths and documented scenarios.

Step 4 - Code Generation: Synthesize your findings into a complete, compilation-ready test suite.

Enclose the final Java code between the [TEST_START] and [TEST_END] tags. Class source code: <source_code>."

Prompt 3: Refactoring:

"You are an expert Java developer. I want to improve the maintainability of the attached C_0 class: <path_class>. Create C_X without changing C_0 functionality and by removing the following smells reported by SonarCloud that you can find in the photo attached to this prompt. Do not include in C_X changes different from what I asked. In fact, consider that C_X should replace C_0 and work with the other components of the system as C_0 currently does. This is an important request; take all the time you need to provide a complete and accurate answer to this request. Class source code: <source_code>."